

Lab Report for ME417 Mechatronics

Laboratory 06

Seyit Kubilay Uluçay

[redacted]

[redacted]

Department of Mechanical Engineering

April 9, 2024

1 Introduction

In this lab assignment, we will create communication between two Arduinos using the I^2C protocol. This protocol is a synchronous, multi-controller/multi-target, single-ended, serial communication bus, commonly used for connecting lower-speed peripheral integrated circuits to processors and microcontrollers such as Arduino over short distances for intra-board communication.
First question of our experiment is
• Question A: what are the main differences between I2C and SPI protocols.
Answer:

- **I^2C uses two wires one for SDA and other is SCL while SPI requires at least four wires for MISO, MOSI, SCK and SS.**
- **I^2C can either send or receive data at a time but SPI can send and receive data simultaneously.**
- **I^2C supports multi-master and multi-slave configuration, while SPI typically supports single-master configuration.**

2 Experiments

In this laboratory session, we will hold one main experiment with some particularly completing experiments. First we will implement a basic button to Arduino, secondly we will implement built-in master and slave codes, read them and understand the logic. Lastly we will arrange these codes with our desire to complete our experiments.

2.1 Experiment 1: Implementing button

First we will construct basic button configuration on the breadboard with arudino as shown in the Figure 1.

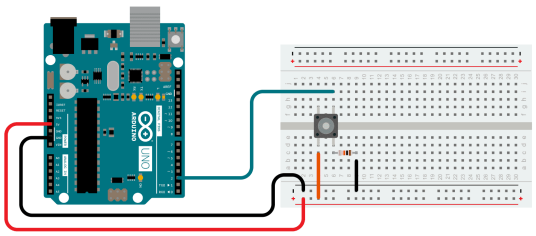


Figure 1: Arduino push-button connection schematic.

In order to construct that circuit we need to use components listed below.

- 1X Breadboard
- 3X Jumper cables
- 1X Push Button
- 1X 10kΩ

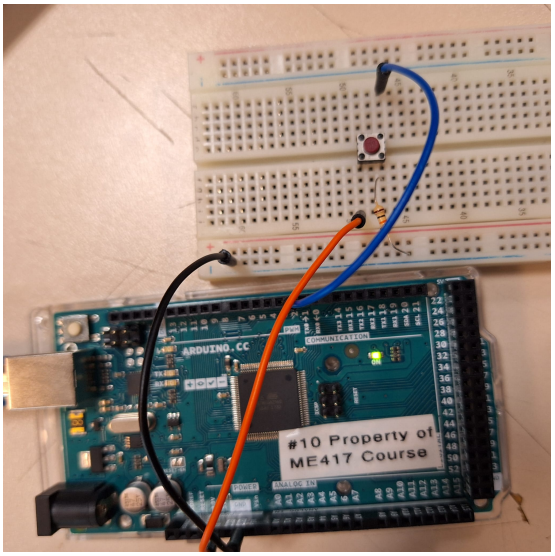


Figure 2: Arduino push-button connection on the breadboard.

After we construct the circuit shown in the Figure 1, we have implemented the Push Button example from: (File − > Examples − > Digital − > Button) as shown in the Figure 2.

```
// constants won't change. They're used here to set pin numbers:
const int buttonPin = 2;  // the number of the pushbutton pin
const int ledPin = 13;    // the number of the LED pin

// variables will change:
int buttonState = 0;  // variable for reading the pushbutton status

void setup() {
  // initialize the LED pin as an output:
  pinMode(ledPin, OUTPUT);
  // initialize the pushbutton pin as an input:
  pinMode(buttonPin, INPUT);
}

void loop() {
  // read the state of the pushbutton value:
  buttonState = digitalRead(buttonPin);

  // check if the pushbutton is pressed. If it is, the buttonState is HIGH:
  if (buttonState == HIGH) {
    // turn LED on:
    digitalWrite(ledPin, HIGH);
  } else {
    // turn LED off:
    digitalWrite(ledPin, LOW);
  }
}
```

Figure 3: Arduino built in push-button code.

Question B: In the code if we do not want to write this line: (const int ledPin = 13;), can we then use the number 13 directly to refer to the Led, as follows: (pinMode(13, OUTPUT); digitalWrite(13, HIGH);)?

Answer: Yes

2.2 Experiment 2: I^2C Communication

In this part we have implemented Master and Slave codes from Arduinos' built in libraries as shown in the Figure 3 and Figure 4.

```
#include <Wire.h>

void setup() {
  Wire.begin(); // join I2C bus (address optional for master)
}

byte x = 0;

void loop() {
  Wire.beginTransmission(8); // transmit to device #8
  Wire.write("x is ");        // sends five bytes
  Wire.write(x);              // sends one byte
  Wire.endTransmission();    // stop transmitting

  x++;
  delay(500);
}
```

Figure 4: Built in master writer code.

```
#include <Wire.h>

void setup() {
  Wire.begin(8);                // join I2C bus with address #8
  Wire.onReceive(receiveEvent); // register event
  Serial.begin(9600);           // start serial for output
}

void loop() {
  delay(100);
}

// function that executes whenever data is received from master
// this function is registered as an event, see setup()
void receiveEvent(int howMany) {
  while (1 < Wire.available()) { // loop through all but the last
    char c = Wire.read(); // receive byte as a character
    Serial.print(c);         // print the character
  }
  int x = Wire.read();    // receive byte as an integer
  Serial.println(x);         // print the integer
}
```

Figure 5: Built in slave receiver code.

• **Question C: The Master and Slave should use the same address to communicate through, which address are the two Arduinos using?**

Answer: The two Arduinos are using the I2C address 4 for communication. This is specified in the Wire.beginTransmission(4); line in the Master code and the Wire.begin(4); line in the Slave code. The number 4 is the address of the Slave device on the I2C bus, and the Master uses this address to initiate communication with the Slave.

2.3 Experiment 3: I^2C Communication with Push Button

In this combined experiment, we will have the Master Arduino transmit a value (the status of the Button) to the Slave Arduino. Upon receiving this value, the Slave Arduino will respond accordingly by either switching the LED on or off.

First we have connected the master and slave arduinos shown as the Figure 6 below.

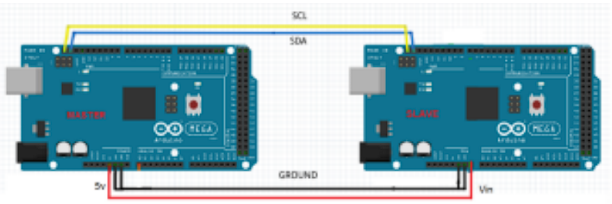


Figure 6: Master and Slave connection.

Then our completed circuit looks like this as shown in the Figure 7.

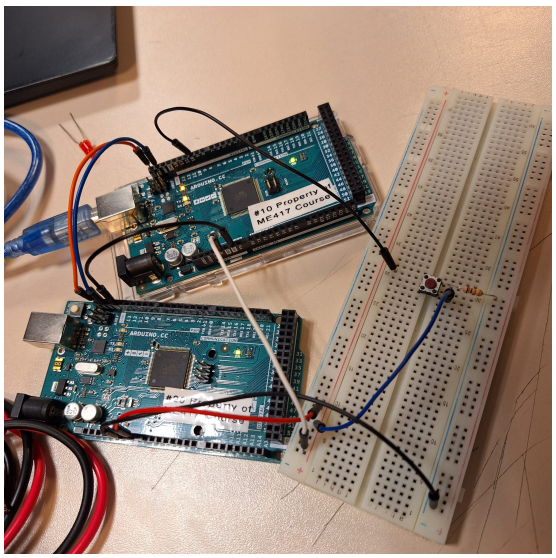


Figure 7: Master and Slave connection on breadboard.

After connections we had to update Master and Slave codes in our favor to make the circuit light up whenever we press the button.

So we have edited Master and Slave codes as shown in the figures down below.

```
#include <Wire.h>

const int buttonPin = 10;

void setup()
{
  Serial.begin(9600);
  Wire.begin(); // join i2c bus (address optional for master)
  pinMode(buttonPin, INPUT);
}

void loop()
{
  int buttonState = digitalRead(buttonPin);
  Wire.beginTransmission(4); // transmit to device #4
  Wire.write("button state is ");        // sends five bytes
  Wire.write(buttonState);              // sends one byte
  Wire.endTransmission();    // stop transmitting
  delay(500);
  Serial.println(digitalRead(buttonPin));
}
```

Figure 8: Master code edited.

```
#include <Wire.h>

const int ledPin = 13;

void setup()
{
  Wire.begin(4);                // join i2c bus with address #4
  Wire.onReceive(receiveEvent); // register event
  pinMode(ledPin, OUTPUT);
}

void loop()
{
  delay(100);
  int c = Wire.read();
  Serial.println(digitalRead(c));

}

void receiveEvent(int howMany)
{
  while (Wire.available()) {
    int c = Wire.read();
    if (c == 1) {
      digitalWrite(ledPin, HIGH);
    } else if (c == 0) {
      digitalWrite(ledPin, LOW);
    }
  }
}
```

Figure 9: Slave code edited.

• **Question D: Edit the code to make the LED blink when we press the button and stop when we release the button. Hint: you only need to edit the Slave code; and we there is an example about LED blink which you can refer to it.**

Answer: In the figure 9 we have edited our code to work as asked in the question, in our circuit LED blinks whenever we press to the button.

3 Conclusion

In conclusion, this lab has provided valuable insights into communication protocols like I^2C in mechatronics. Through practical experiments, we have learned how these protocols works in the real life with how we can code and edit in our favor. This experience provided better understanding in communication between two microcontrollers.